

REPORT

In this project, I developed a comprehensive movie management system designed to efficiently store, retrieve, and analyze movie data using a custom hash table. My main goal was to handle large datasets seamlessly while providing quick access and detailed information about each movie.

To start, I implemented the `readFile` method to parse a CSV file containing movie details like title, genre, release year, IMDb ID, rating, and platform availability. Each record is carefully processed, and if a movie with the given IMDb ID doesn't exist in the hash table, I created a new `Records` object and stored it. This ensures that every movie is uniquely identified and properly organized. I also built a flexible transaction system within the `Records` class to manage platform and country-specific details, which are sorted for better presentation.

One of the most critical parts of the system was designing the hash table. I integrated options to adjust the load factor dynamically and choose between different hashing methods—SSF or PAF—and collision resolution strategies like linear probing or double hashing. This flexibility allows the system to adapt to varying dataset sizes and optimize performance. I also added methods to evaluate hash table performance, such as collision counting and average search time calculations.

To enhance functionality, I included features to search for movie details by IMDb ID, list movies available in specific countries, and identify titles streaming across all platforms. A particular highlight was implementing batch search operations with detailed performance metrics, which allowed me to track found and missing records efficiently.

I paid attention to modularity throughout the project by creating reusable components like the `Records` and `Transaction` classes. Each class encapsulates specific responsibilities—movies and their attributes in `Records`, and platform or availability details in `Transaction`. Sorting mechanisms in the `Records` class ensure that data is always displayed in an organized manner.

Overall, this system is a balance of efficiency, flexibility, and clarity. It handles large datasets effectively, provides detailed insights into movie records, and ensures smooth performance even under high demand.

Load Factor	Hash Function	Collision Handling	Collision Count	Indexing Time	Avg. Search Time
$\alpha=50\%$	SSF	LP	3854460721	202114125801	1563674.0
		DH	8721712	5474237400	19461.0
	PAF	LP	76941	3526237300	1482.0
		DH	98185	3123451201	958.0
$\alpha=80\%$	SSF	LP	5703112332	264633218200	1399032.0
		DH	14515653	6192349800	18821.0
	PAF	LP	458794	3526251800	968.0
		DH	221323	3933263400	1441.0

```

--- Media Program Menu ---
1. Load dataset
2. Run 1000 search test
3. Search for a media item with the ImdbId
4. List the top 10 media according to user votes
5. List all the media streams in a given country
6. List the media items streaming on all 5 platforms
7. Exit
Enter your choice: 3
Enter ImdbId: tt0266697
Choice for load factor: 1-) 0.5 2-) 0.8
2
Choice for hash function: 1-) SSF 2-) PAF
2
Choice for handling method: 1-) Linear Probing 2-) Double Hashing
2

>Search: tt0266697

Type: movie
Genre: Action, Crime, Thriller
Release Year: 2003
Imdb ID: tt0266697
Rating: 8.2
Number of Votes: 1220488

3 Transactions found for Kill Bill: Vol. 1

Netflix, AE, AL, AO, AT, AU, AZ, BG, BH, BY, CA, CI, CM, CZ, DE, DK, EE, EG, FI, FJ, GH, GQ, GR, HR, HU,
Apple TV+, BR, CL, CO, MX
Amazon Prime, AG, AR, BB, BM, BO, BR, BS, CL, CO, CR, DO, EC, GT, HN, IN, JM, LC, MX, PA, PE, PY, SV, TC,

CollusionCount :14339403
Index time: 6739237500

Average time: 74266.0

```

Anil CAN